



Machine Learning in Production

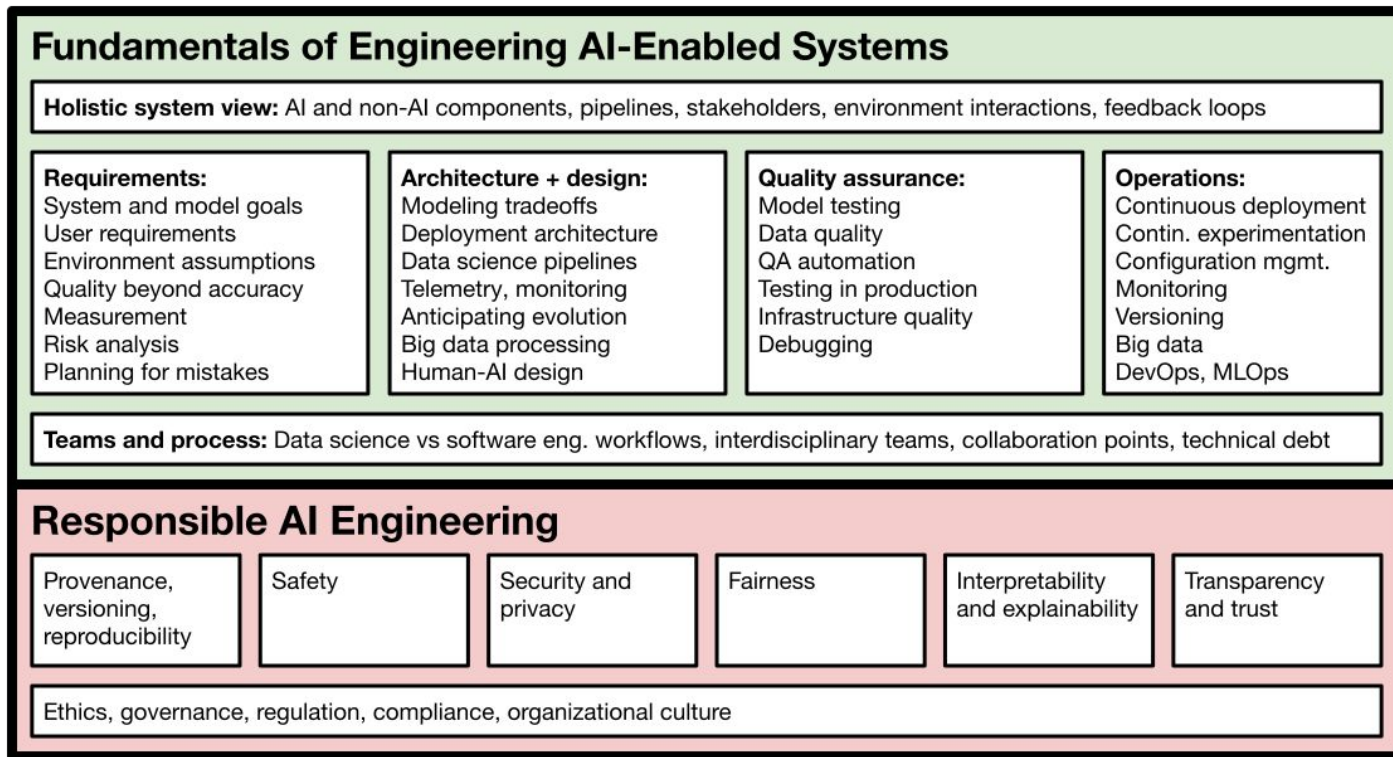
Deploying a Model

Administrativa

Last day to change participation opt-in
(confirm status with ``/participation status``)

All teams should have their VMs
Requests are coming in right now
Security is your responsibility

Exploring architecture and design...



Learning Goals

- Understand important quality considerations when deploying ML components
- Follow a design process to explicitly reason about alternative designs and their quality tradeoffs
- Gather data to make informed decisions about what ML technique to use and where and how to deploy it
- Understand the power of design patterns for codifying design knowledge
- Create architectural models to reason about relevant characteristics
- Critique the decision of where an AI model lives (e.g., cloud vs edge vs hybrid), considering the relevant tradeoffs
- Deploy models locally and to the cloud
- Document model inference services

Deploying a Model is Easy

Deploying a Model is Easy

Model inference component as function/library

```
from sklearn.linear_model import LogisticRegression
model = ... # learn model or load serialized model ...
def infer(feature1, feature2):
    return model.predict(np.array([[feature1, feature2]]))
```

Deploying a Model is Easy

Model inference component as a service

```
from flask import Flask, escape, request
app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = '/tmp/uploads'
detector_model = ... # load model...

# inference API that returns JSON with classes
# found in an image
@app.route('/get_objects', methods=['POST'])
def pred():
    uploaded_img = request.files["images"]
    converted_img = ... # feature encoding of uploaded img
    result = detector_model(converted_img)
    return jsonify({"response":
                    result['detection_class_entities']})
```

Deploying a Model is Easy

LLM wrapper as a service

```
app = Flask(__name__)
client = anthropic.Anthropic()

PROMPT = """You are a {role}. Summarize in
{n} sentences. ... Text: {text}"""

@app.route('/summarize', methods=['POST'])
def pred():
    p = PROMPT.format(role="news editor", n=2,
                      text=request.json["text"])
    result = client.messages.create(
        model="claude-sonnet-5", max_tokens=200,
        messages=[{"role": "user", "content": p}])
    return jsonify({"response": result.content[0].text})
```

Deploying a Model is Easy

Packaging a model inference service in a container

```
FROM python:3.8-buster
RUN pip install uwsgi==2.0.20
RUN pip install tensorflow==2.7.0
RUN pip install flask==2.0.2
RUN pip install gunicorn==20.1.0
COPY models/model.pf /model/
COPY ./serve.py /app/main.py
WORKDIR ./app
EXPOSE 4040
CMD ["gunicorn", "-b 0.0.0.0:4040", "main:app"]
```

Deploying a Model is Easy

Self-hosting an open-weights LLM on GPU

```
FROM vllm/vllm-openai:v0.11.0
CMD ["--model", "Qwen/Qwen3-8B",
     "--tensor-parallel-size", "2",    # split over 2 GPUs
     "--max-model-len", "32768",      # context window
     "--gpu-memory-utilization", "0.9",
     "--enable-prefix-caching",
     "--port", "8000"]
```

Deploying a Model is Easy

Model inference component as a service in the cloud

- Package in container or other infrastructure
- Deploy in cloud infrastructure, optionally with GPUs
- Auto-scaling with demand ("*Stateless Serving Functions Pattern*")
- MLOps infrastructure to automate all of this (more on this later)
 - [BentoML](#) (low code service creation, deployment, model registry),
 - [Cortex](#) (automated deployment and scaling of models on AWS),
 - [TFX model serving](#) (tensorflow GRPC services)
 - [Seldon Core](#) (no-code model service and many many additional services for monitoring and operations on Kubernetes)

But is it really easy?

Offline use?

Deployment at scale?

Hardware needs and operating cost?

Frequent updates?

Integration of the model into a system? (pipelines, RAG)

Long running tasks? (agents)

Meeting system requirements?

Every system is different!

What qualities do we want from a model deployment?



Every System is Different

Personalized music recommendations for Spotify

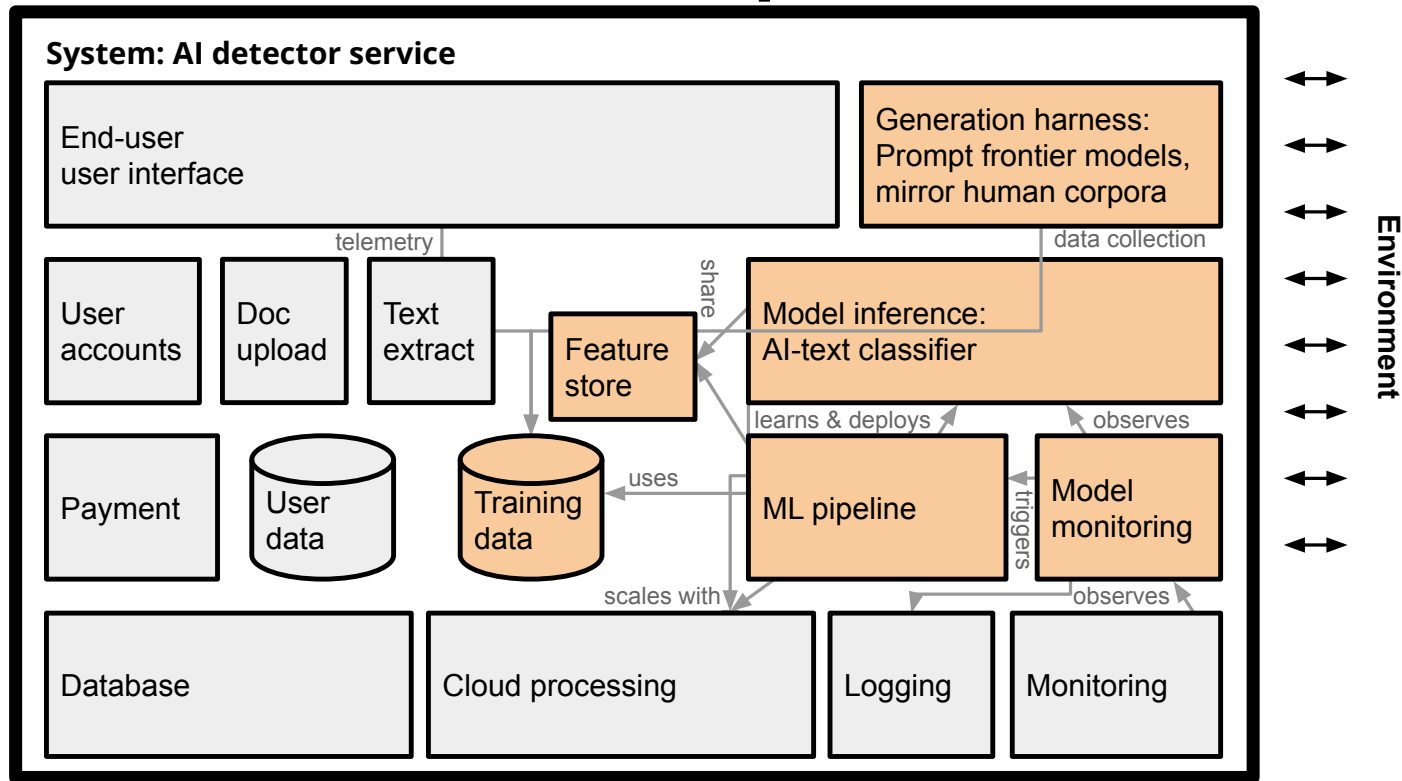
Cancer prognosis

AI detection startup

Agentic travel receipt processing

Smart keyboard for mobile device

Inference is a Component within a System



Legend: Non-ML component, ML component, system boundary

Case Study: Augmented Reality Translation



Case Study: Augmented Reality Translation



Case Study: Augmented Reality Translation



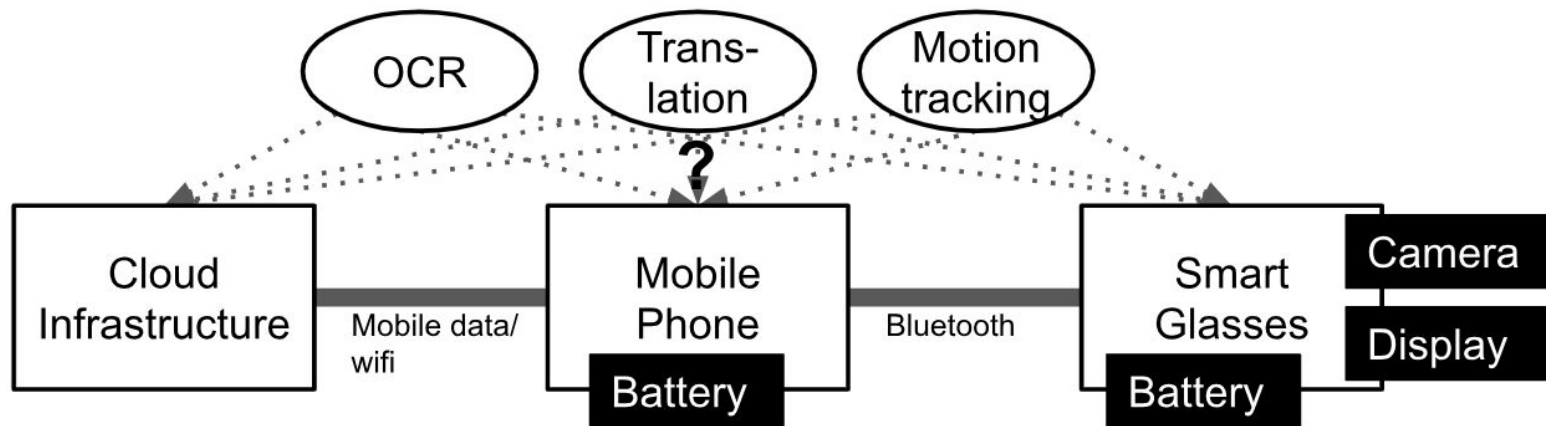
Case Study: Augmented Reality Translation



System Qualities of Interest?



Where Should the Models Live?



Cloud? Phone? Glasses?

What qualities are relevant for the decision?

Considerations

How much data is needed as input for the model?

How much output data is produced by the model?

How fast/energy consuming is model execution?

What latency is needed for the application?

How big is the model? How often does it need to be updated?

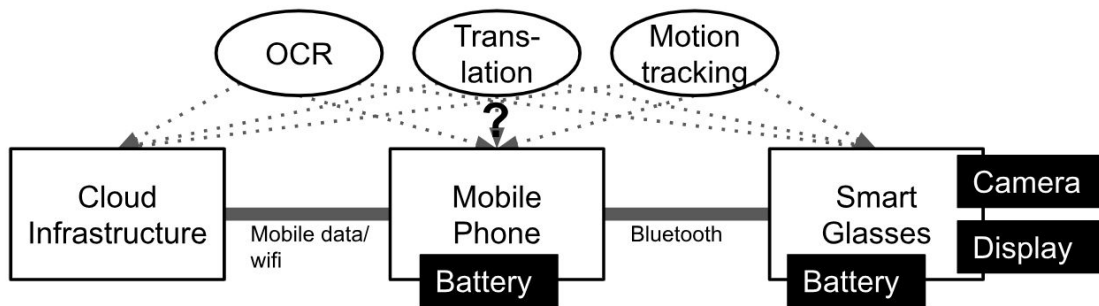
Cost of operating the model? (distribution + execution)

Opportunities for telemetry?

What happens if users are offline?

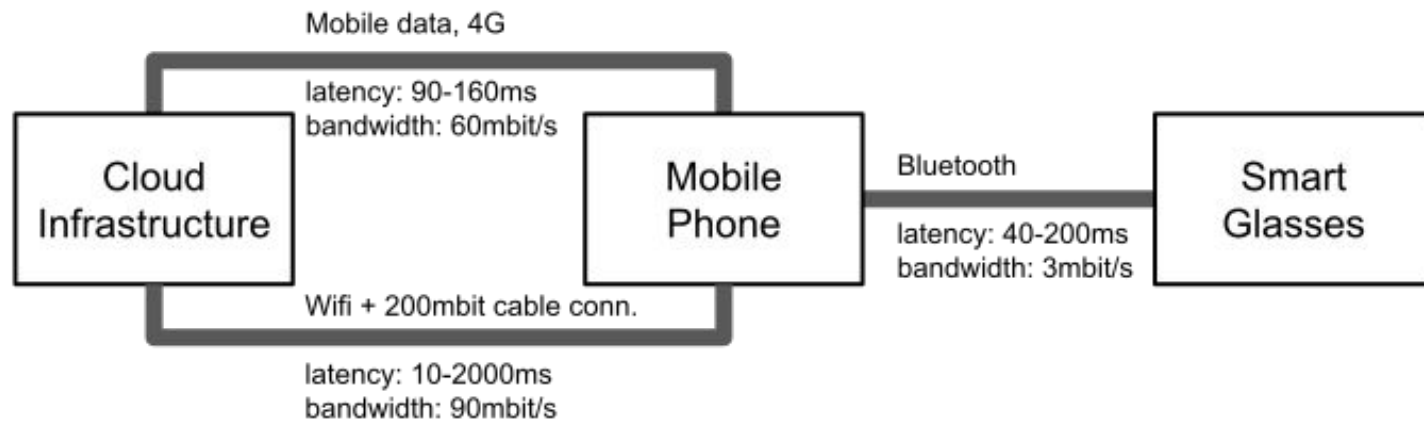
Breakout: Latency and Bandwidth Analysis

1. Estimate latency and bandwidth requirements between components
2. Discuss tradeoffs among different deployment models

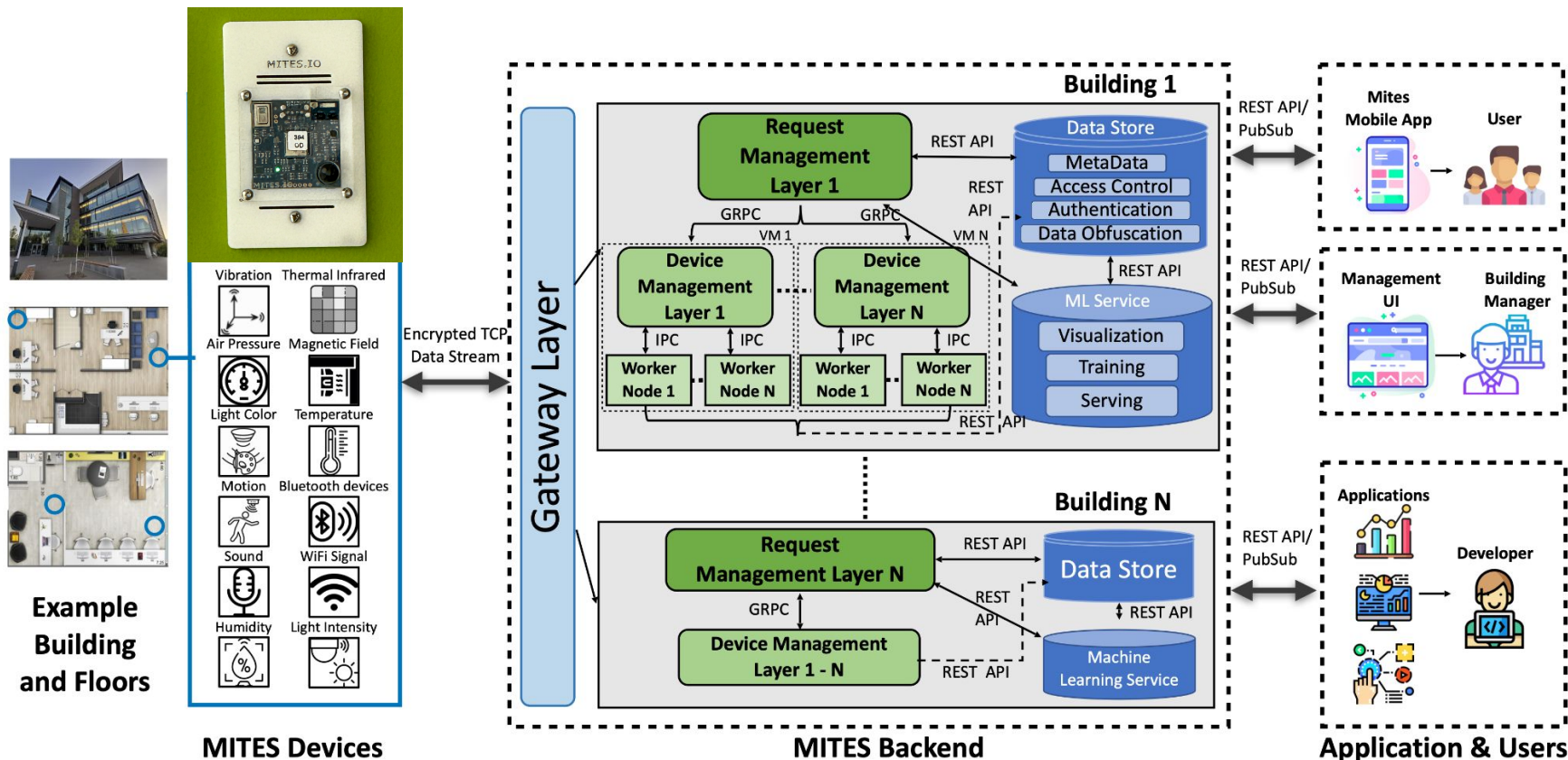


As a group, post and follow up in #lecture, tagging group members and @mlipbot:

- Recommended deployment for OCR (with justification):
- Recommended deployment for Translation (with justification):



“Edge” Computing: Mites in TCS



Excursion: Architectural Modeling and Reasoning

Architectural Expertise as Differentiator?

As code becomes cheap to produce...

- Easy to build the wrong system fast...

- Easy to build prototype, deploy model, but will it scale?

- Boosts senior engineers who steer AI to good architecture

Architecture requires larger thinking and planning, judgement, taste

See also Russinovich & Hanselman, CACM, April 2026

Machine Learning in Production • Christian Kaestner & Bogdan Vasilescu, Carnegie Mellon University • Fall 2026

Architectural concerns

Buy, train, or prompt a model? Cost-benefit tradeoffs

What deployment is feasible with 10k monthly users? Cost-accuracy tradeoffs

Cloud vs on-prem vs on-device deployment? One large model or many small ones? Latency-cost-privacy-stability tradeoffs

Linear pipeline, agent, or RAG? Cost-capability-security tradeoff

What breaks when model provider updates or is not available?

What data leaves your trust boundary?

Who owns the safeguards? Who controls versioning?

Architectural concerns

Buy, train, or prompt a model? Cost-benefit tradeoffs

What deployment is feasible with 10k monthly users? Cost-accuracy tradeoffs

Cloud vs on-prem vs on-device deployment? One large model or many small ones? Latency-cost-privacy-stability tradeoffs

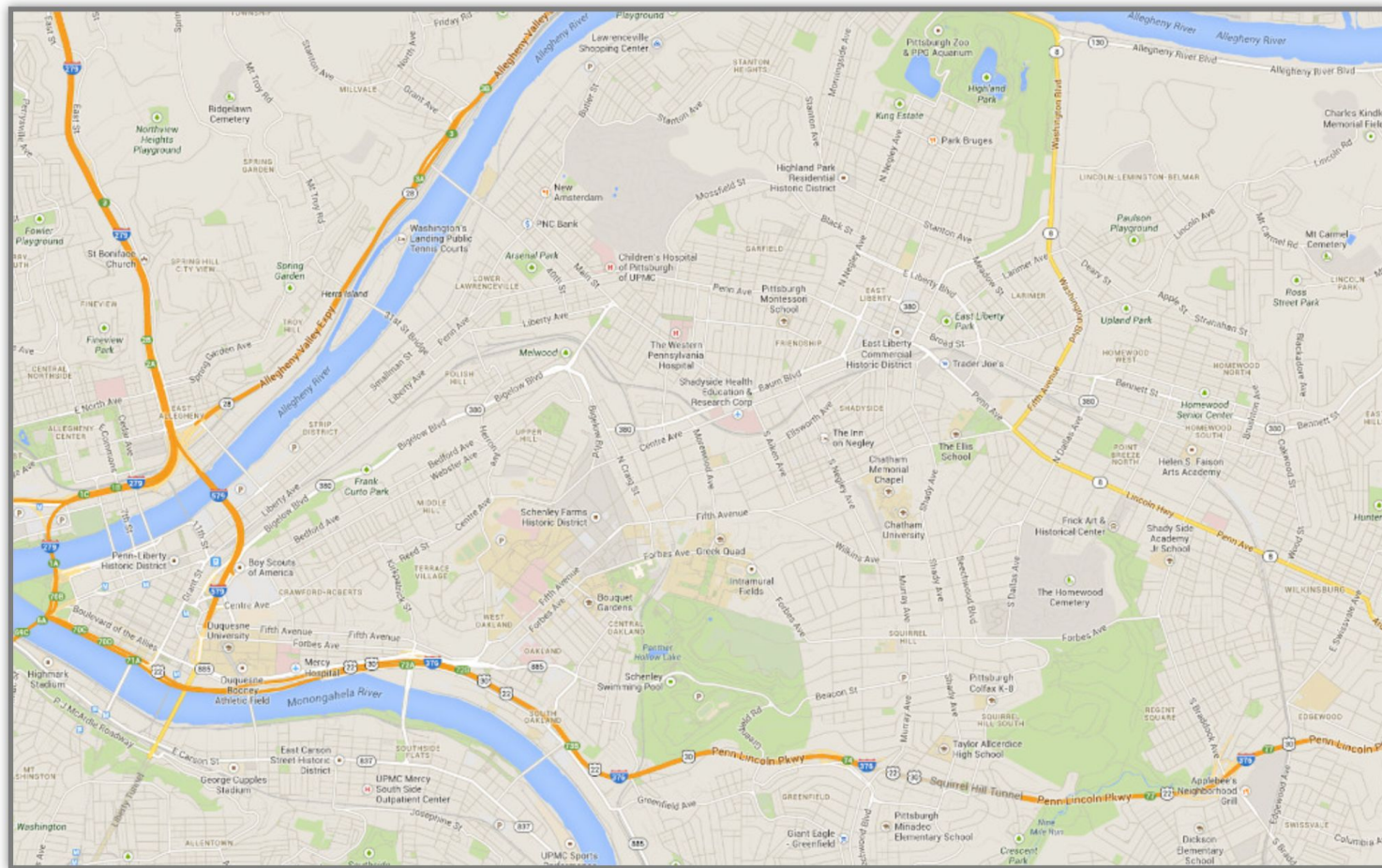
Linear pipeline, agent, or RAG? Cost-capability-security tradeoff

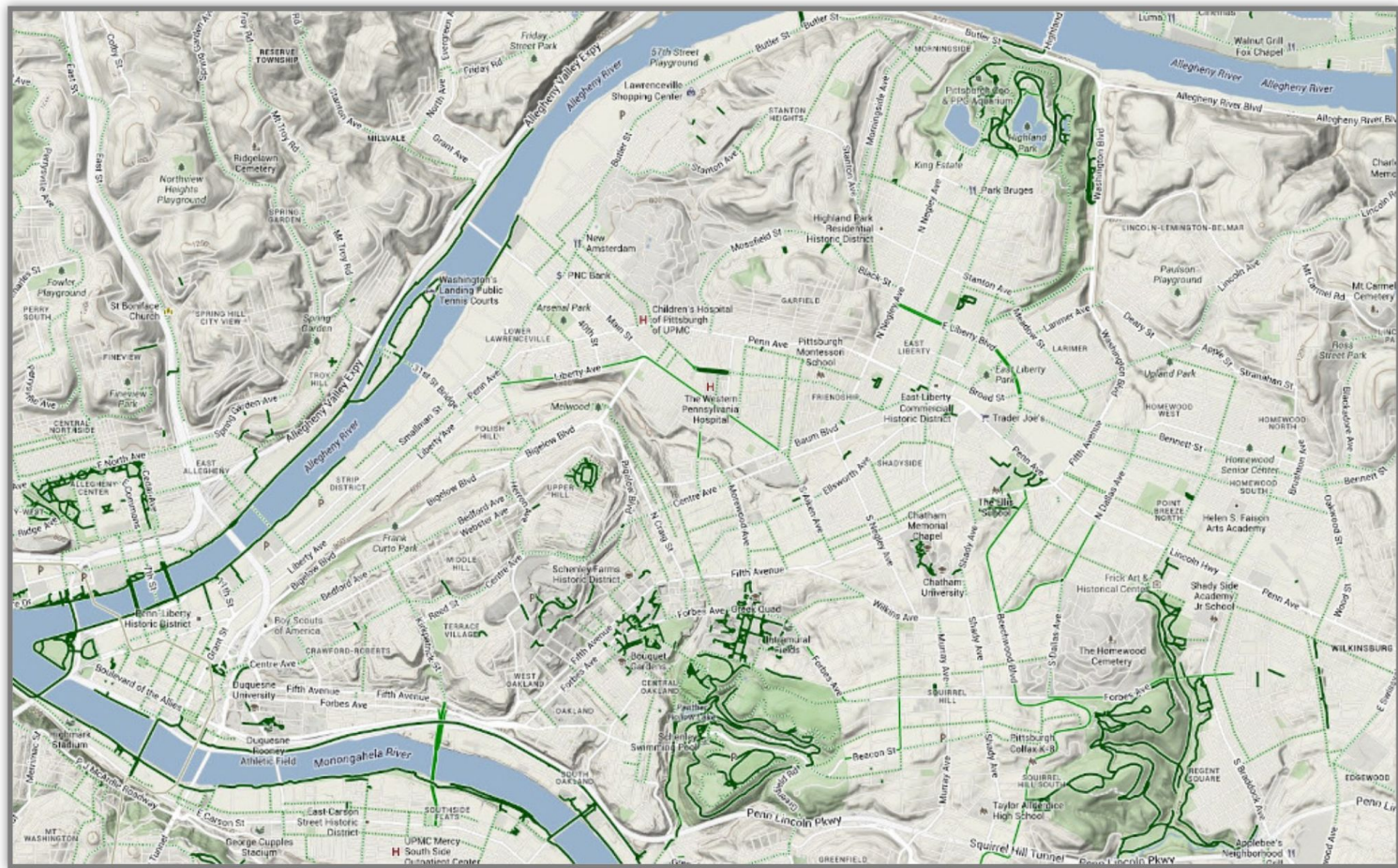
What breaks when model provider updates or is not available?

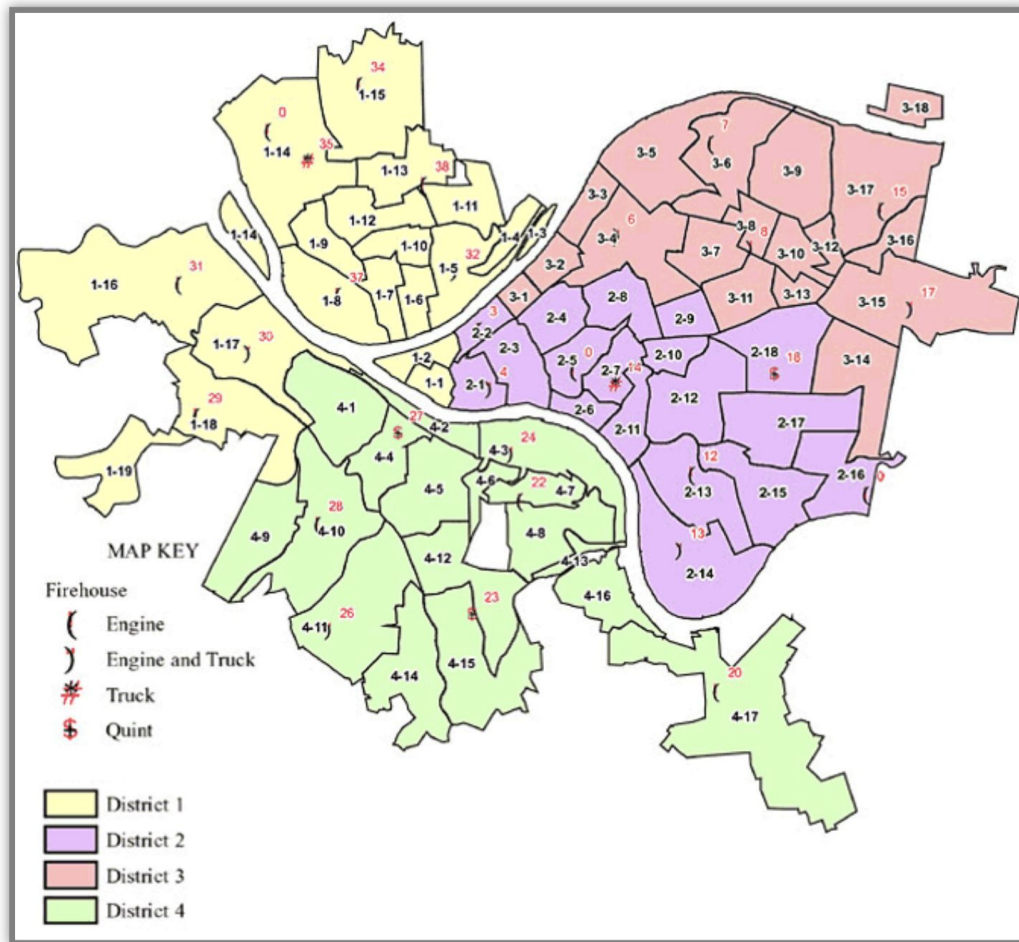
What data leaves your trust boundary?

Who owns the safeguards? Who controls versioning?

Movie recommendations?







Analysis-Specific Abstractions

All maps were abstractions of the same real-world construct

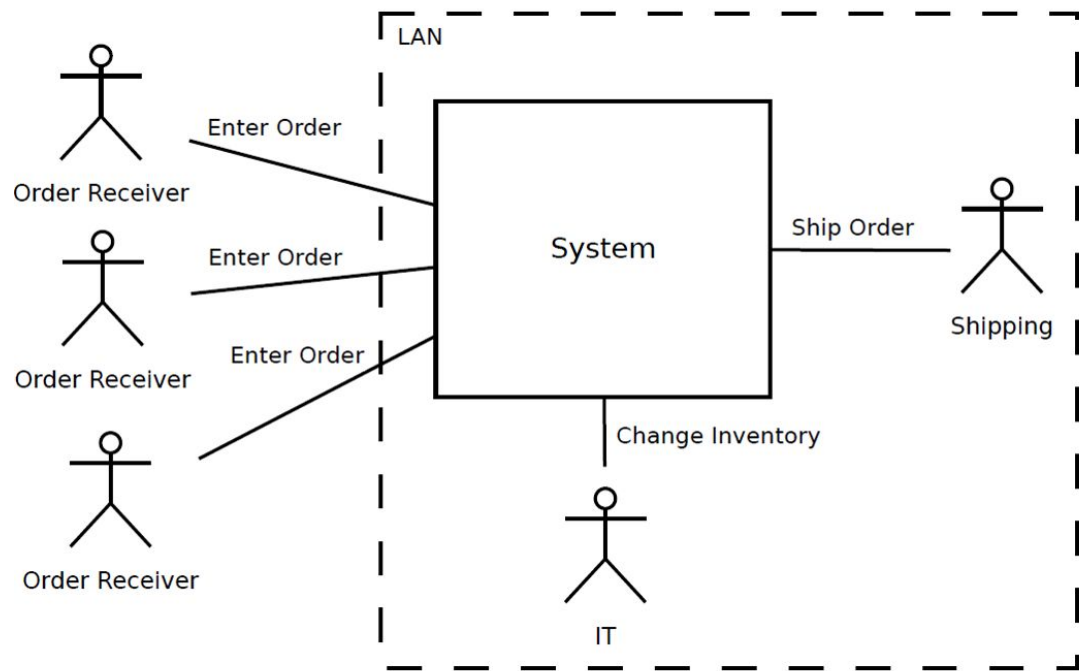
All maps were created with different goals in mind

- Different relevant abstractions
- Different reasoning opportunities

Architectural models are specific system abstractions, for reasoning about specific qualities

No uniform notation

What can we reason about?



What can we reason about?

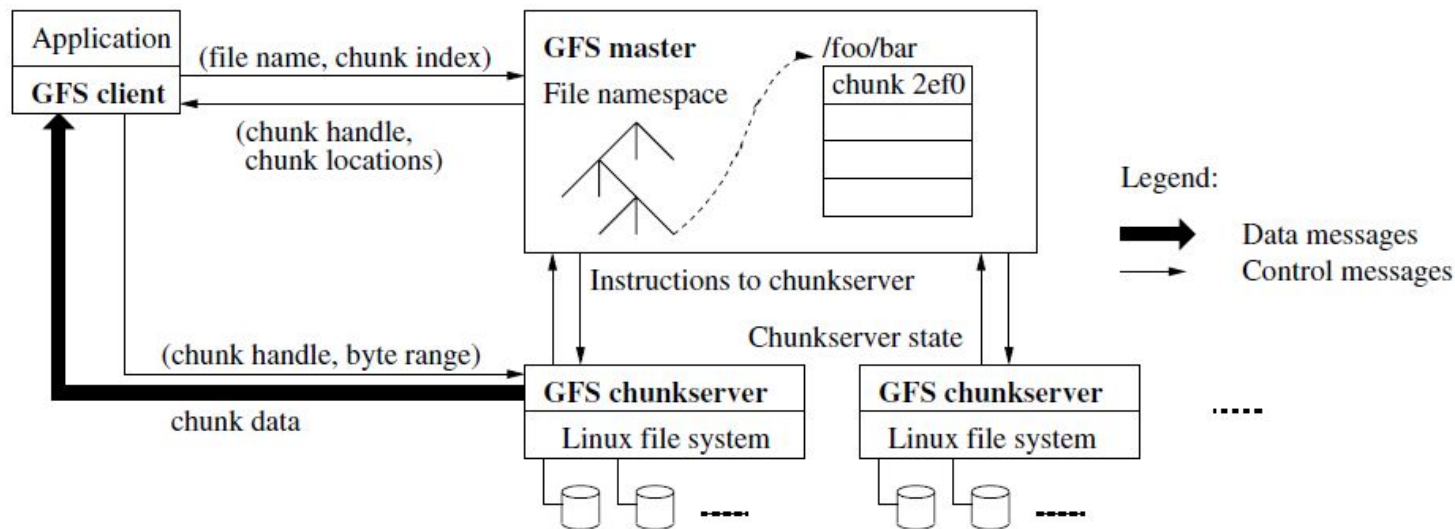
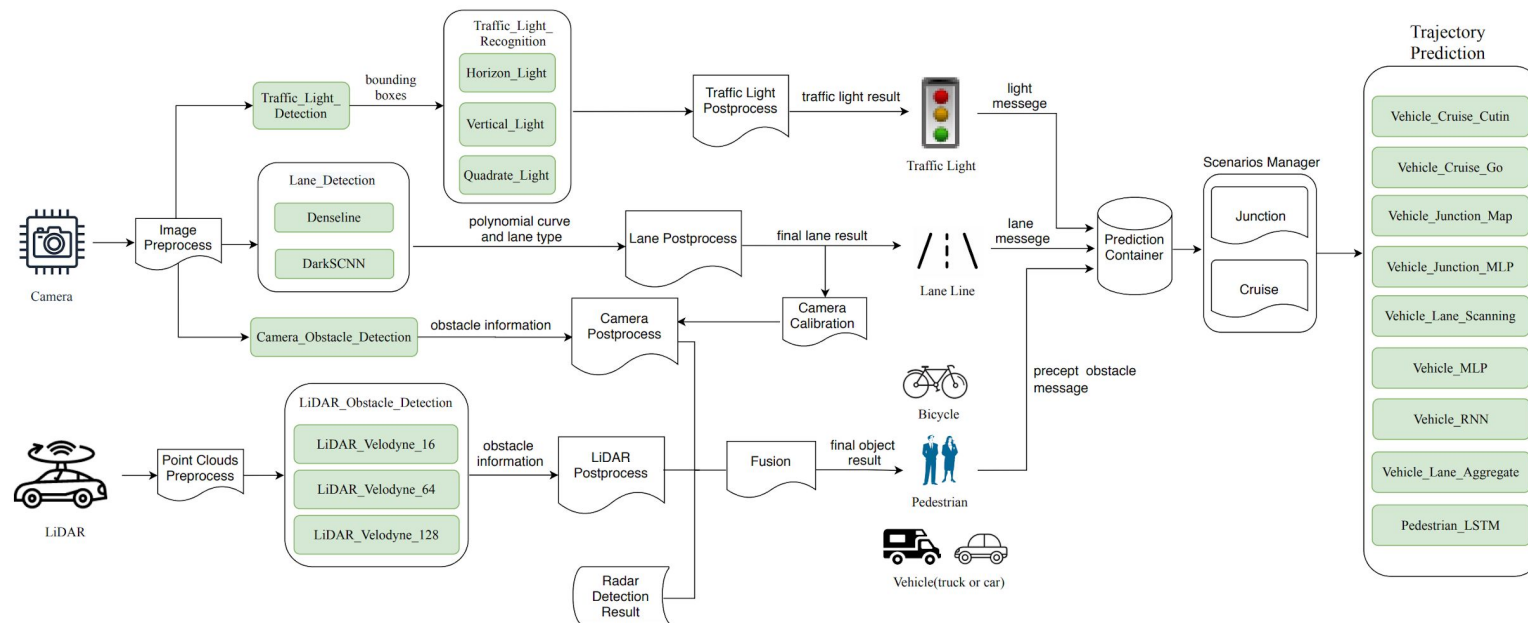


Figure 1: GFS Architecture

What can we reason about?



Peng, Zi, Jinqiu Yang, Tse-Hsun Chen, and Lei Ma. "A first look at the integration of machine learning models in complex autonomous driving systems: a case study on Apollo." In Proc. FSE, 2020.

Suggestions for Graphical Notations

Use notation suitable for analysis

Document meaning of boxes and edges in legend

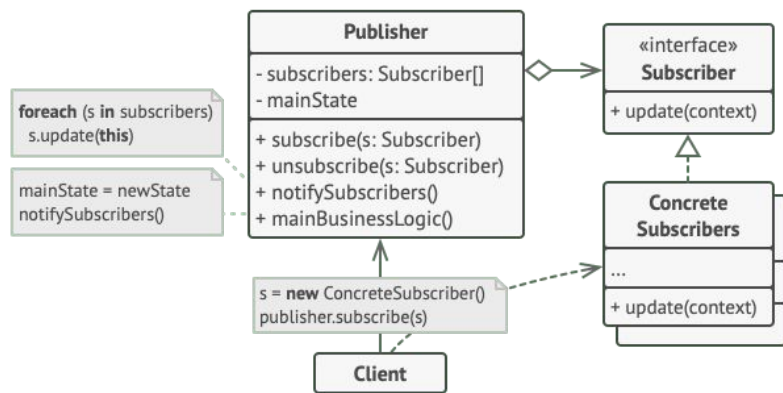
Graphical or textual both okay; whiteboard sketches often sufficient

Formal notations available

Basic Common Deployment Patterns

Design Patterns are Codified Design Knowl.

Vocabulary of design problems and solutions



Example: *Observer pattern* object-oriented design pattern describes a solution how objects can be notified when another object changes without strongly coupling these objects to each other

Agents are well versed in common patterns

Common Architectural Patterns (for traditional non-ML systems)

Client-server architecture

Multi-tier architecture

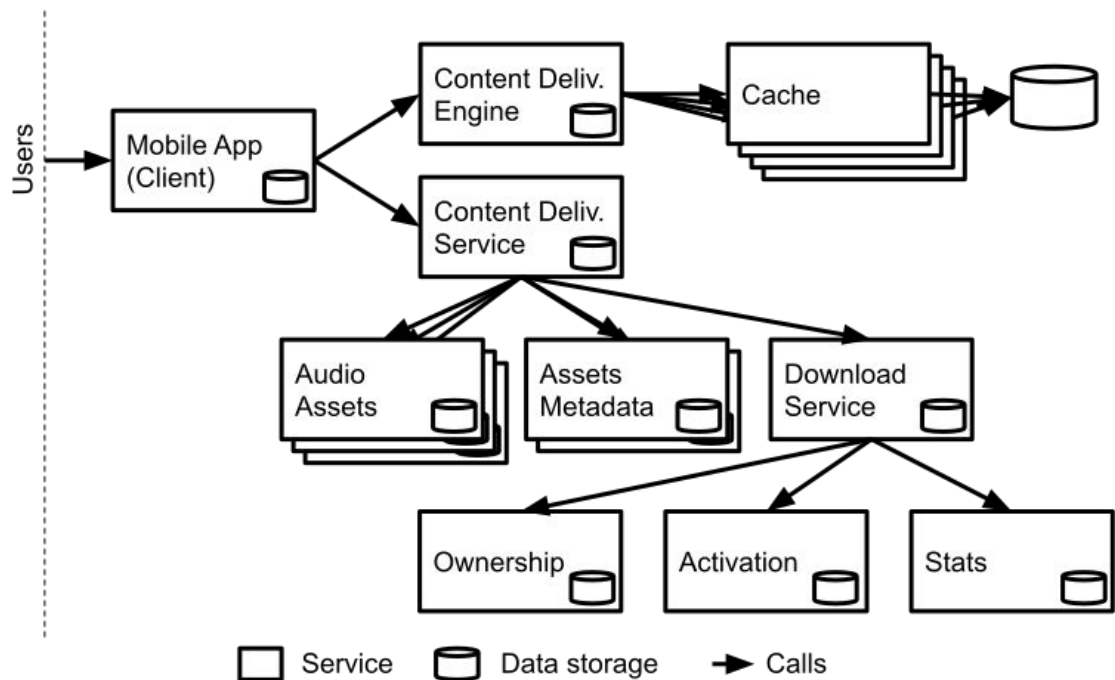
Service-oriented architecture and microservices

Event-based architecture

Data-flow architecture

Micro-service architecture

Microservices



Common Deployment Designs

- Static intelligence in the product
- Client-side intelligence (user-facing devices)
- Server-centric intelligence
- Back-end cached intelligence
- Hybrid models

Consider: Offline use, inference latency, model updates, application updates, operating cost, scalability, protecting intellectual property

Common Deployment Designs

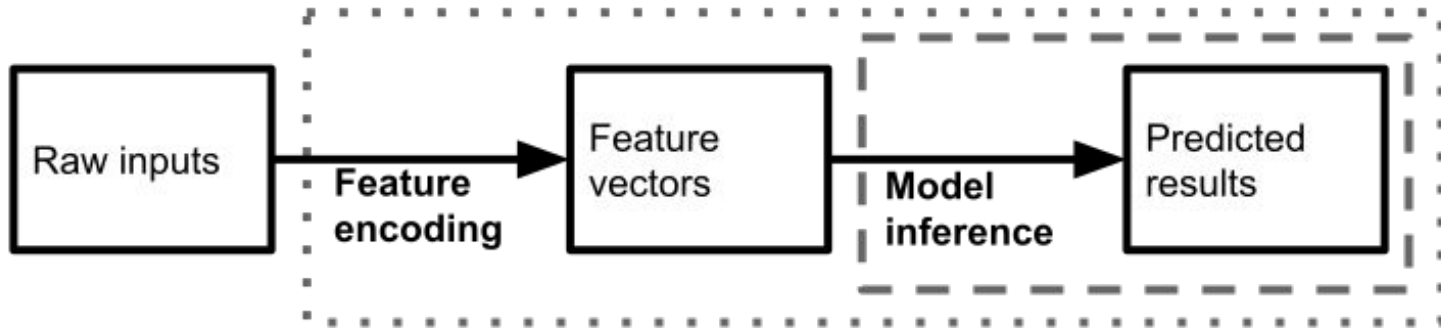
- Static intelligence in the product
 - difficult to update
 - good execution latency
 - cheap operation
 - offline operation
 - no telemetry to evaluate and improve
- Client-side intelligence
 - updates costly/slow, out of sync problems
 - complexity in clients
 - offline operation, low execution latency
- Server-centric intelligence
 - latency in model execution (remote calls)
 - easy to update and experiment
 - operation cost
 - no offline operation
- Back-end cached intelligence
 - precomputed common results
 - fast execution, partial offline
 - saves bandwidth, complicated updates

Another cold-start problem

How long for the first prediction when launching a container?

Loading large models into GPU memory may take a while...

Where Should Feature Encoding Happen?



Should feature encoding happen server-side or client-side? Trade-offs?

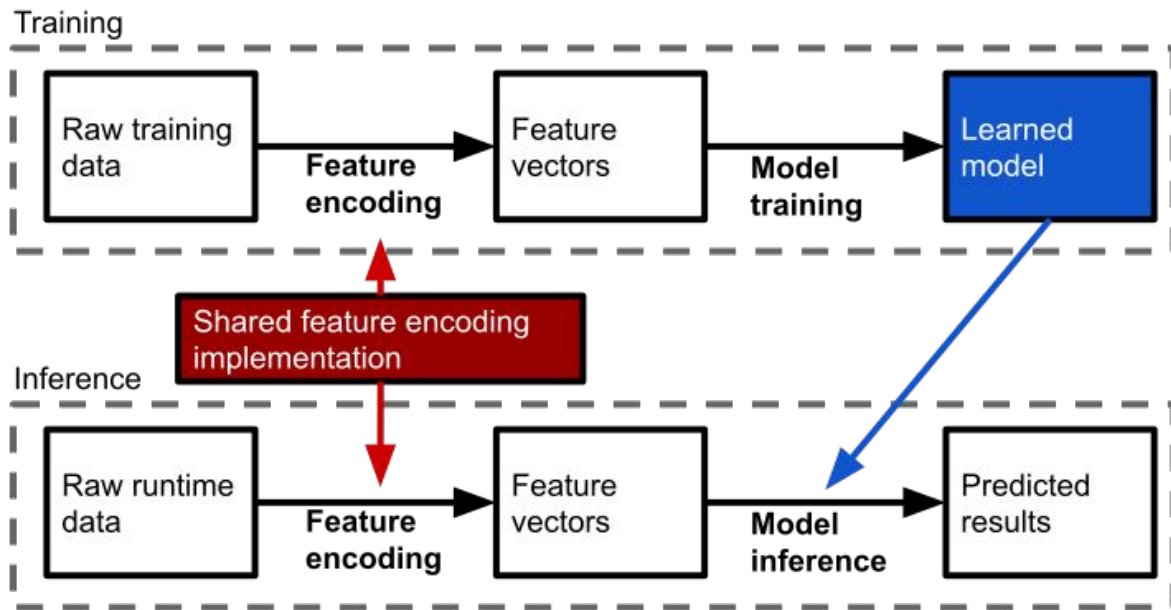
Deploying a Model is Easy

Model inference component as a service

```
from flask import Flask, escape, request
app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = '/tmp/uploads'
detector_model = ... # load model...

# inference API that returns JSON with classes
# found in an image
@app.route('/get_objects', methods=['POST'])
def pred():
    uploaded_img = request.files["images"]
    converted_img = ... # feature encoding of uploaded img
    result = detector_model(converted_img)
    return jsonify({"response":
                    result['detection_class_entities']})
```

Reusing Feature Engineering Code



Avoid training-serving skew

The Feature Store Pattern

- Central place to store, version, and describe feature engineering code
- Can be reused across projects
- Possible caching of expensive features

Many open source and commercial offerings, e.g., Feast, Tecton, AWS SageMaker Feature Store

Patterns for ML-Enabled Systems

- Stateless/serverless Serving Function Pattern
- Feature-Store Pattern
- Batched/precomputed serving pattern
- Two-phase prediction pattern
- Batch Serving Pattern
- Decouple-training-from-serving pattern

Anti-Patterns

- Big Ass Script Architecture
- Dead Experimental Code Paths
- Glue code
- Multiple Language Smell
- Pipeline Jungles
- Plain-Old Datatype Smell
- Undeclared Consumers

Compound AI Systems

Modern ML Systems beyond a Single Model

Still provide a simple interface: input -> answer, but with complex internals:

- Prompt + LLM

- Feature engineering, guardrails, and other processing

- Multiple prompts in a sequence and routing

- Retrieval augmented generation (RAG)

- Tool-use agents, subagents

- Memory

Caching

Cache full response to a prompt? Precompute outputs for common inputs?

Semantic caching for similar responses?

[tools] [system] [patterns] [examples] || [this claim]
└────────── cached prefix ─────────┘ └── pays full ─┘

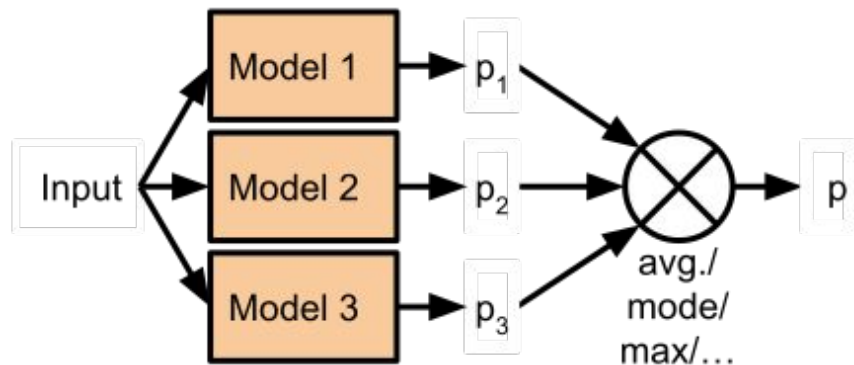
Prefix caching to avoid repeat computations for large prompts and in chat mode

- Lower cost of cached tokens, faster response

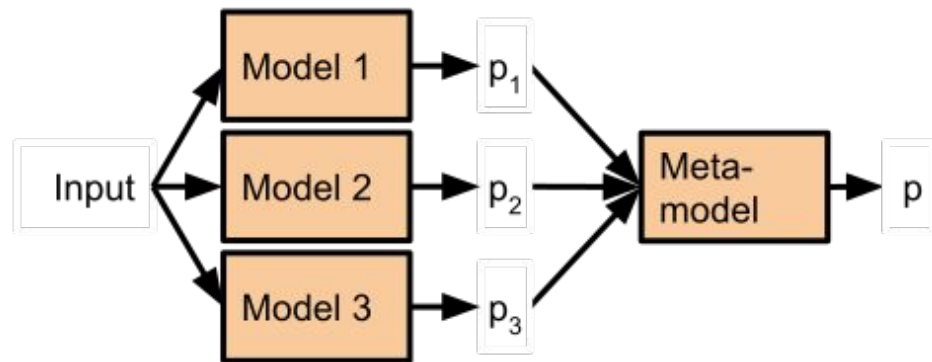
- At provider side, possibly locally with low-level LLM access

Composing Models: Ensemble and Metamodels

Ensemble

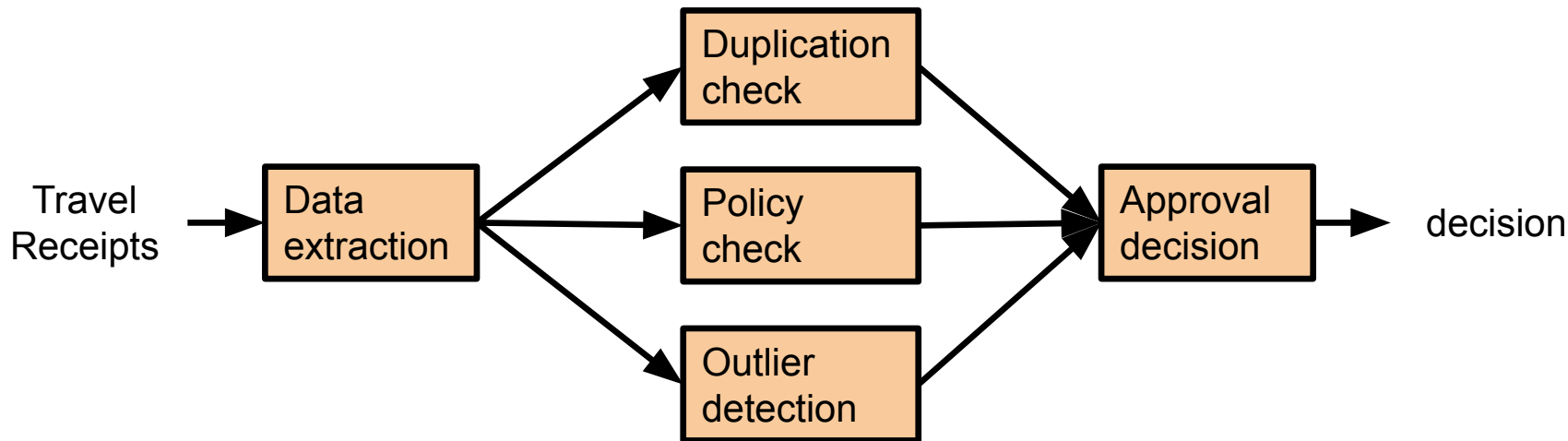


Metamodel / model stacking

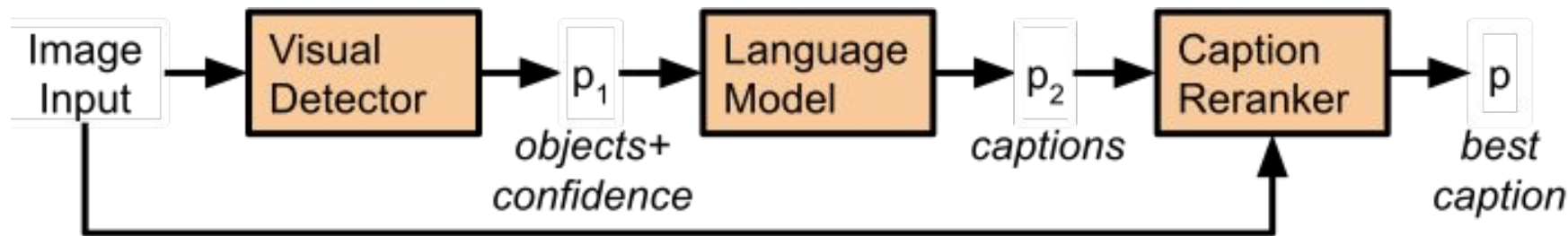


Legend:  machine-learned model,  non-ML aggregation function,  p prediction

Variation: Parallel work and aggregate

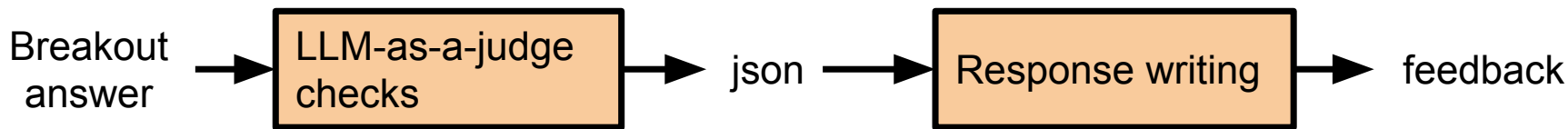


Composing Models: Decomposing the problem, sequential



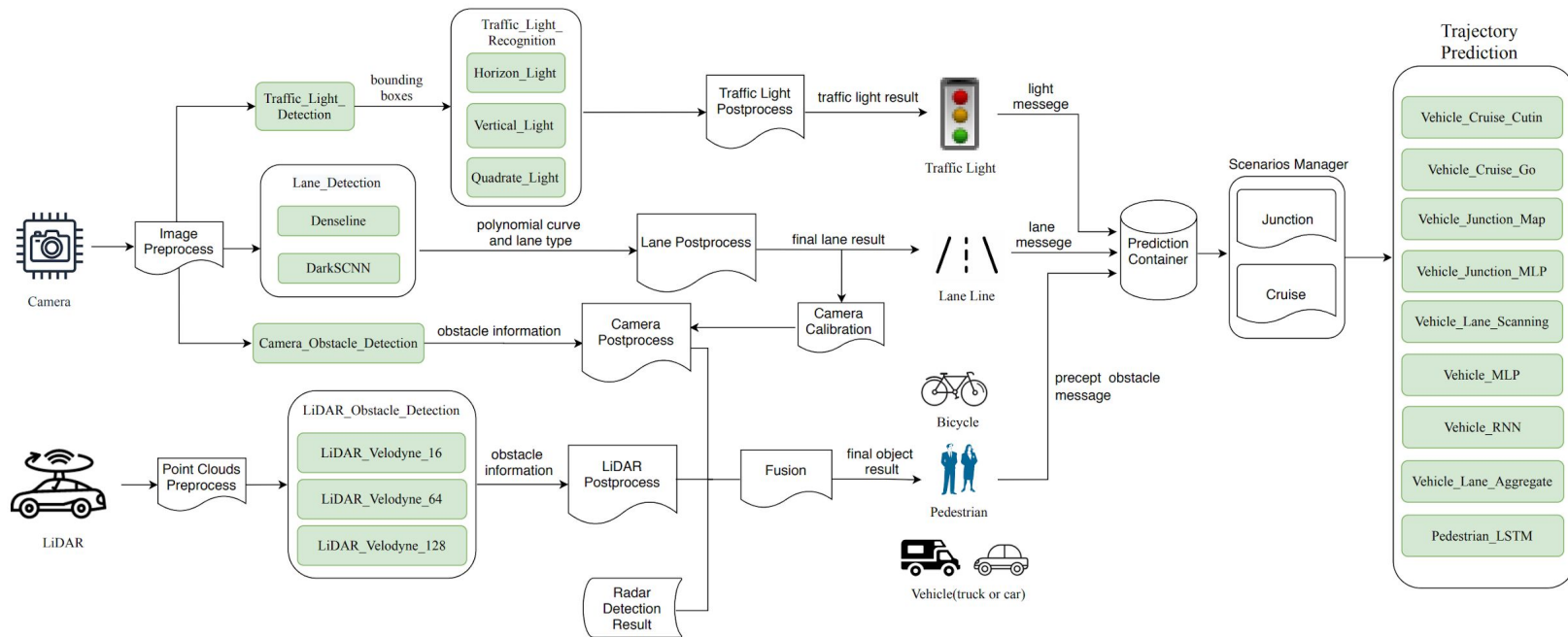
Different models or prompts
specialize in different aspects

Composing Models: Decomposing the problem, sequential

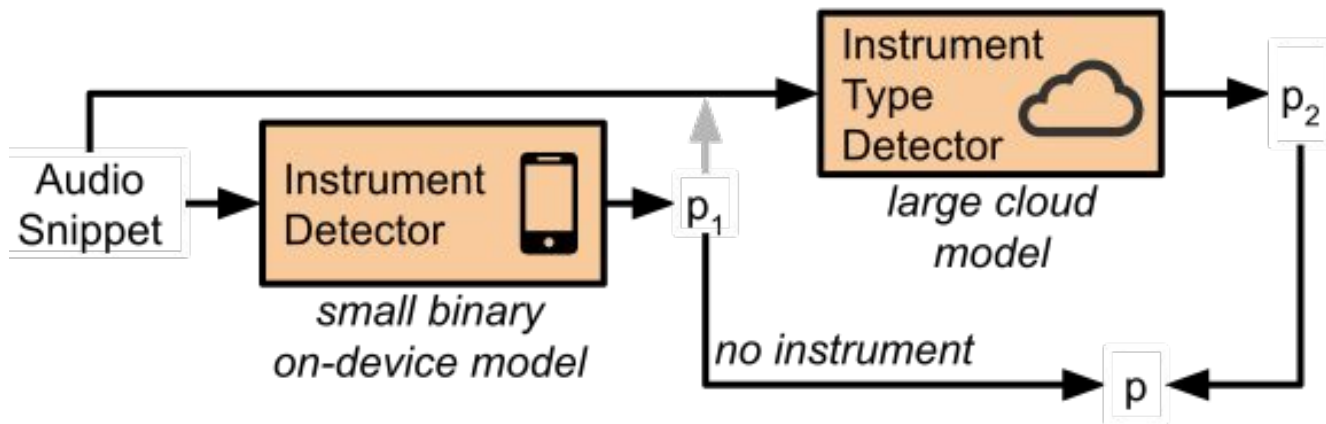


Different models or prompts
specialize in different aspects

Real-Time Serving; Many Models



Composing Models: Cascade/two-phase prediction



Small Language Models (SLM)

Increasing interest in smaller, specialized models

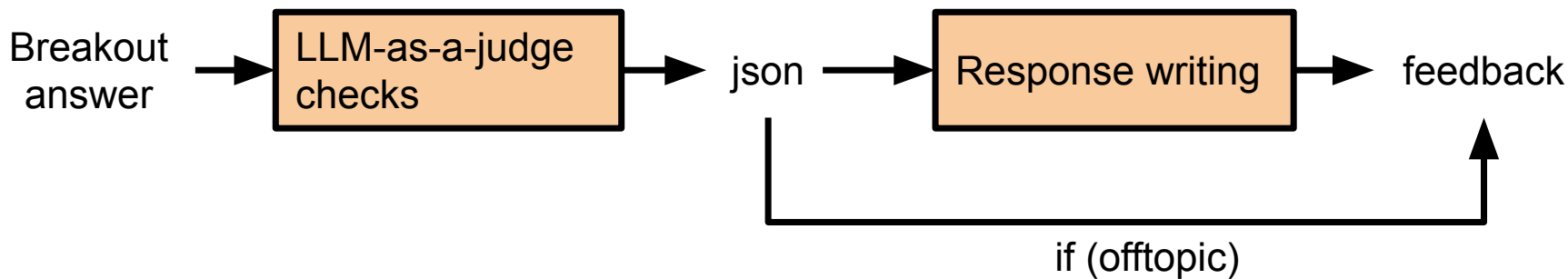
Cheaper to run, faster answers, on-prem deployment

In agents, tailored harnesses can make up for many weaknesses for specific problems

Tradeoffs between frontier vs small language models (engineering effort, runtime cost, accuracy, ...)

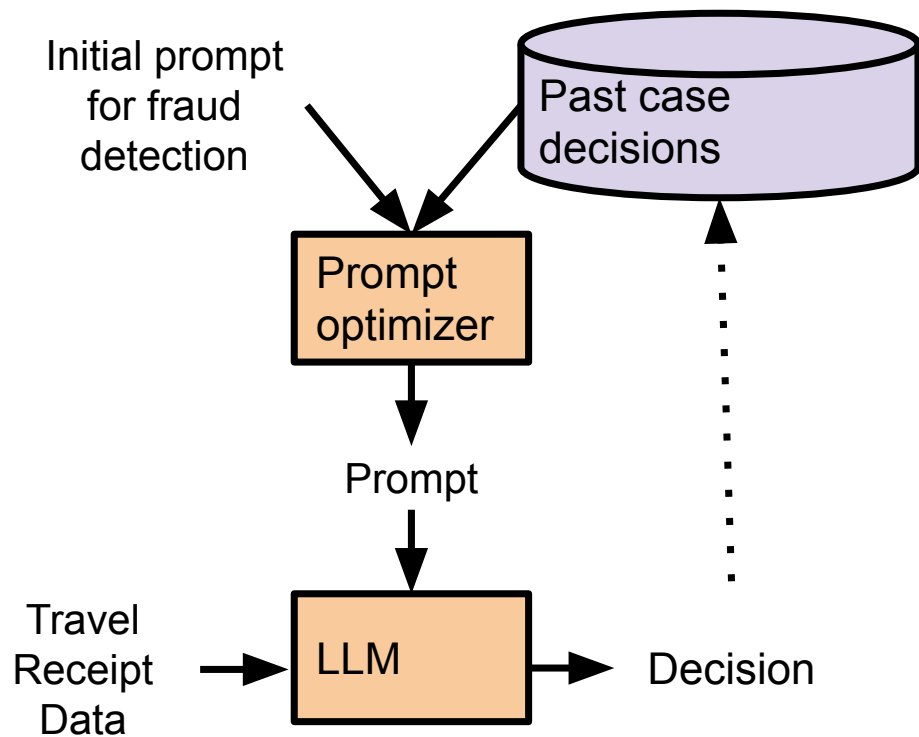
Belcak, Peter, et al. "Small language models are the future of agentic ai." *arXiv preprint arXiv:2506.02153* (2025).

Composing Models: Decomposing the problem, sequential



Different models or prompts
specialize in different aspects

Prompt optimization

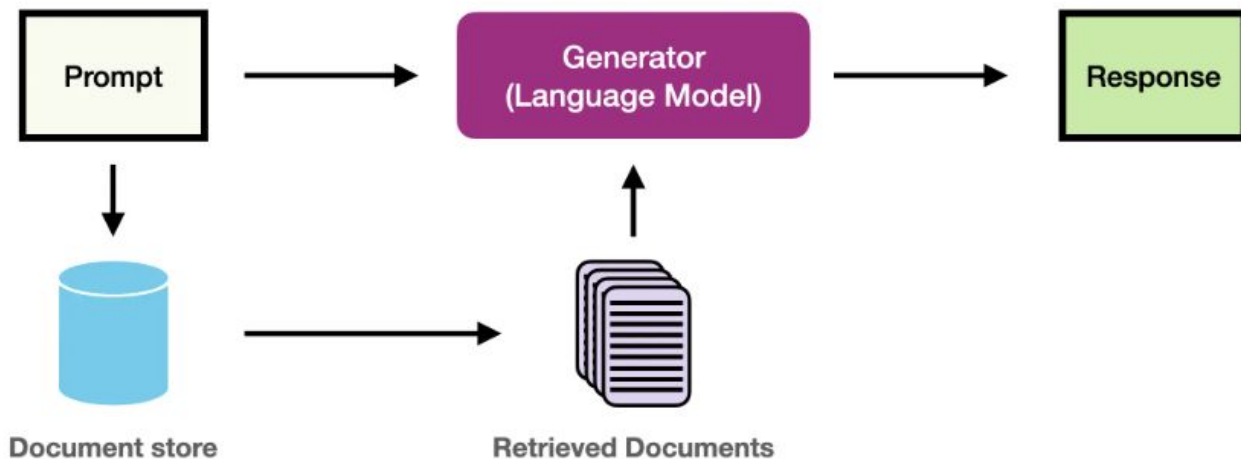


e.g., dspy

Can also
optimize over
chains

Composing Models:

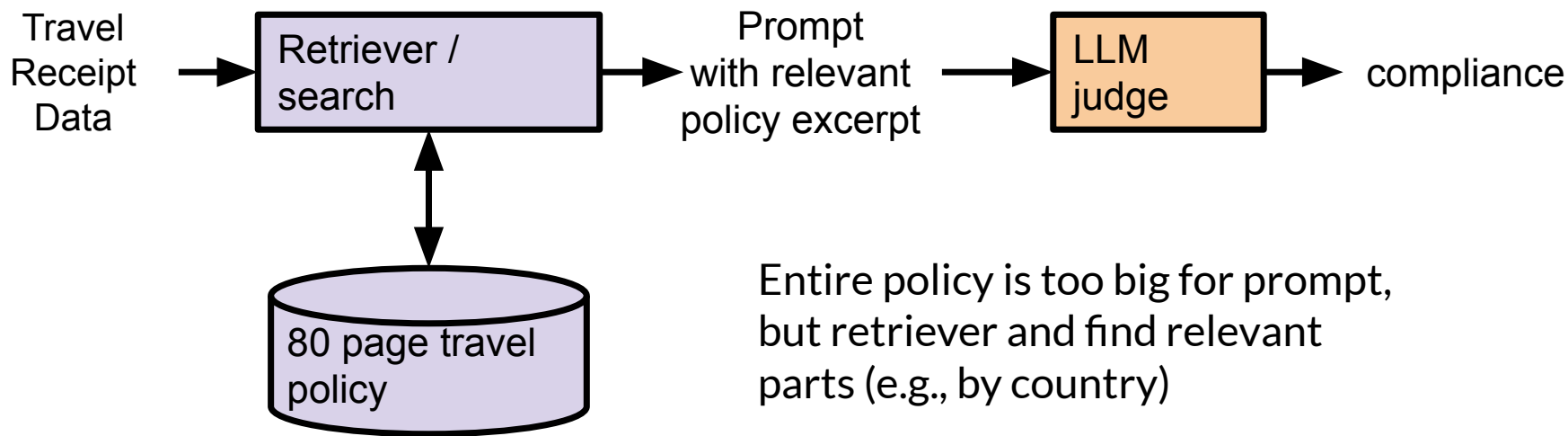
Retrieval Augmented Generation (RAG)



<https://www.promptingguide.ai/research/rag>

Composing Models:

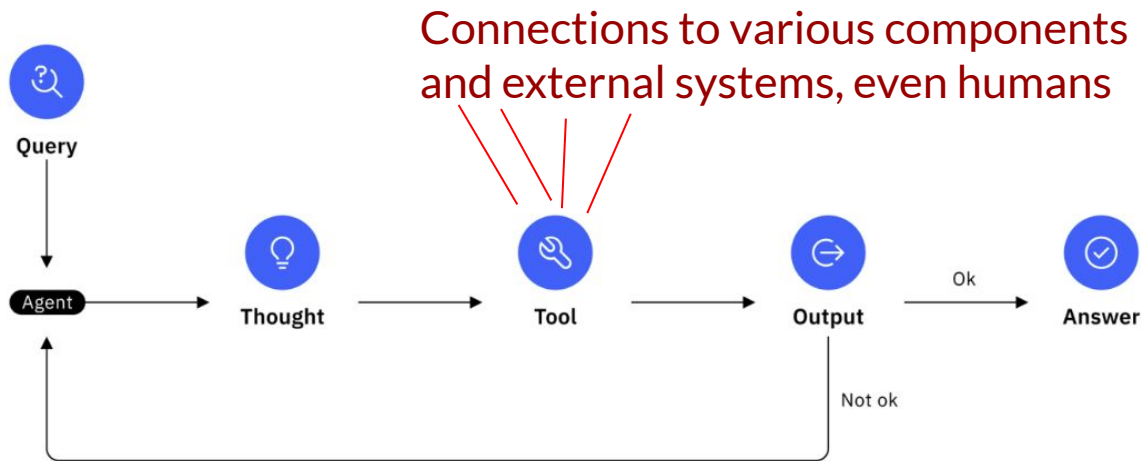
Retrieval Augmented Generation (RAG)



Tool-use agents

```
def react_agent(query):  
    context = llm(f"Think: How to answer '{query}'?")  
    while not is_final_answer(context):  
        action = llm(f"Context: {context}\nAction:")  
        if "get_email" in action:  
            result = get_email(extract_date_range(action))  
        elif "send_email" in action:  
            result = send_email(extract_to_body(action))  
        context = llm(f"Previous: {context}\nAction: {action}\nResult: {result}\nContext:")  
    return llm(f"Final answer based on: {context}")
```

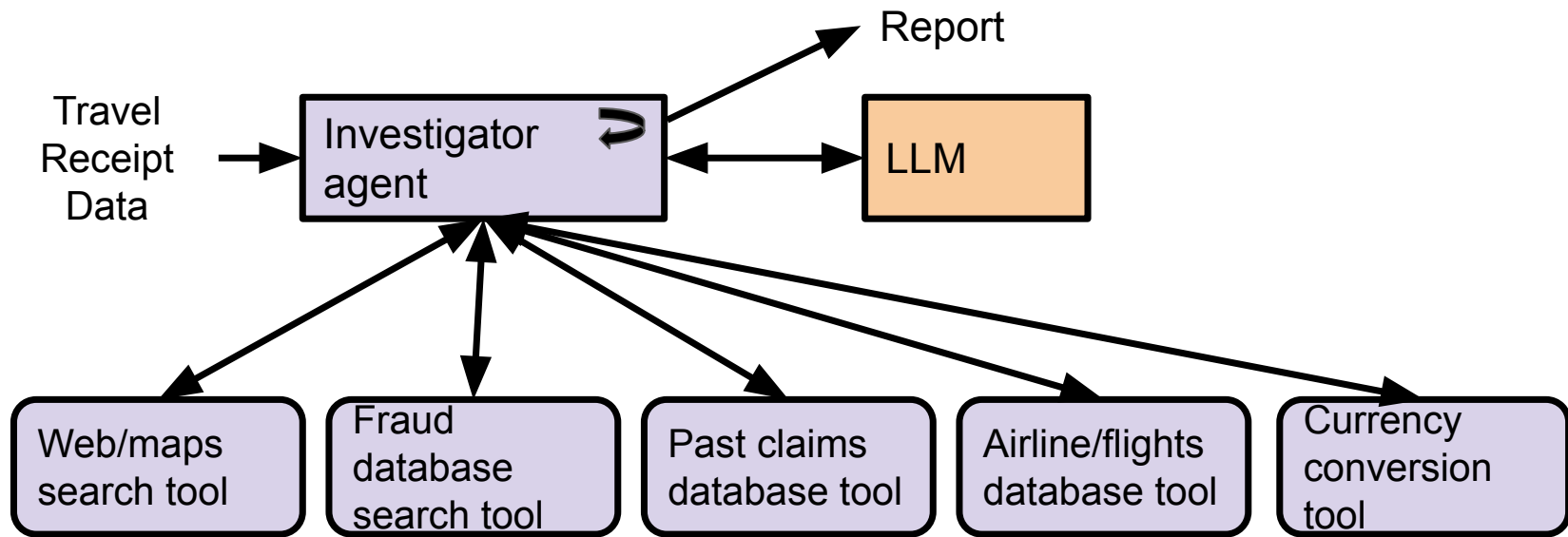
Inversion of control(!)



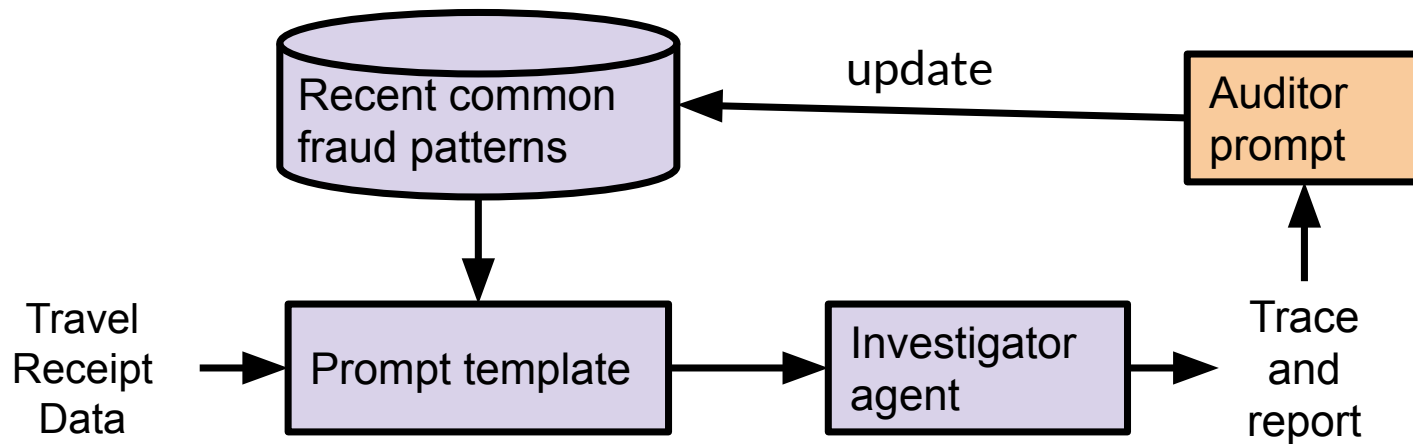
<https://www.ibm.com/think/topics/react-agent>

Tool-use agents

Creative tool use to interact with environment

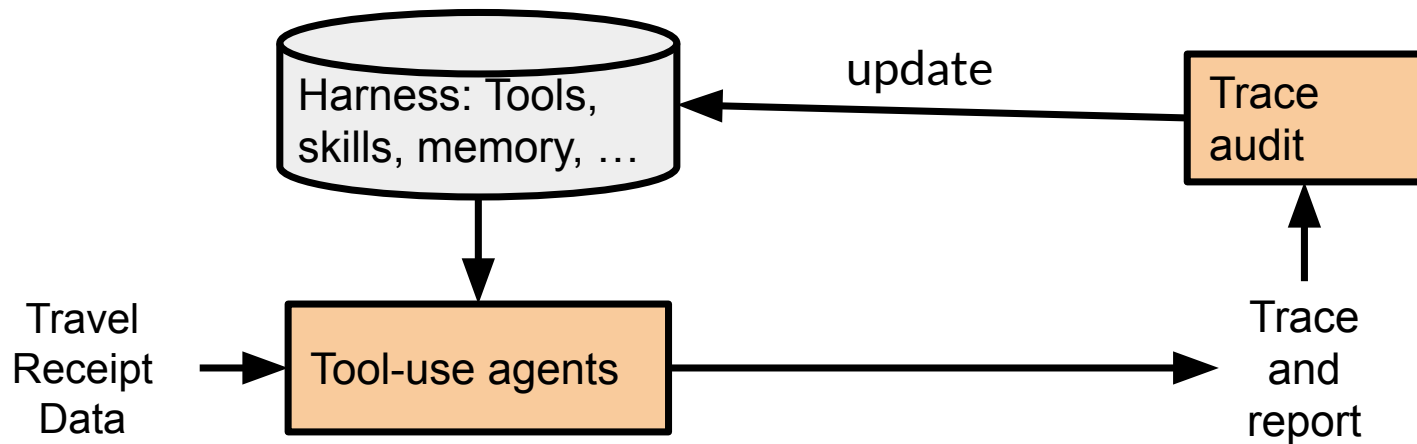


Memory



Update state based on each/some executions

Automated agent harness adaptation



Update also tools, prompts, and skills

Many of these are well known architectural patterns

Chaining is a pipeline

Routing is a broker

Orchestrator and workers are leader-worker systems

Evaluator and optimizer is a control loop

Compound AI Systems change Architecture Considerations

Memory, caching undermine stateless assumptions

- more complex deployments
- risks of malicious attacks

LLMs, sequencing and agents can provide extensive latency

- from services to event-based systems

Operational Robustness

Redundancy for availability?

Load balancer for scalability?

Can mistakes be isolated?

- Local error handling?
- Telemetry to isolate errors to component?

Logging and log analysis for what qualities?

Many considerations

Buy, host, train decisions for each model separately

Small vs large model

Qualities beyond accuracy: inference cost, latency (incl. first token latency and inter-token latency), GPU needs, token costs, bandwidth, privacy

Breakout 2: Sketch an Architecture

You try to automate most support emails for an airline. 40,000 emails per day. Decide: response or escalate.

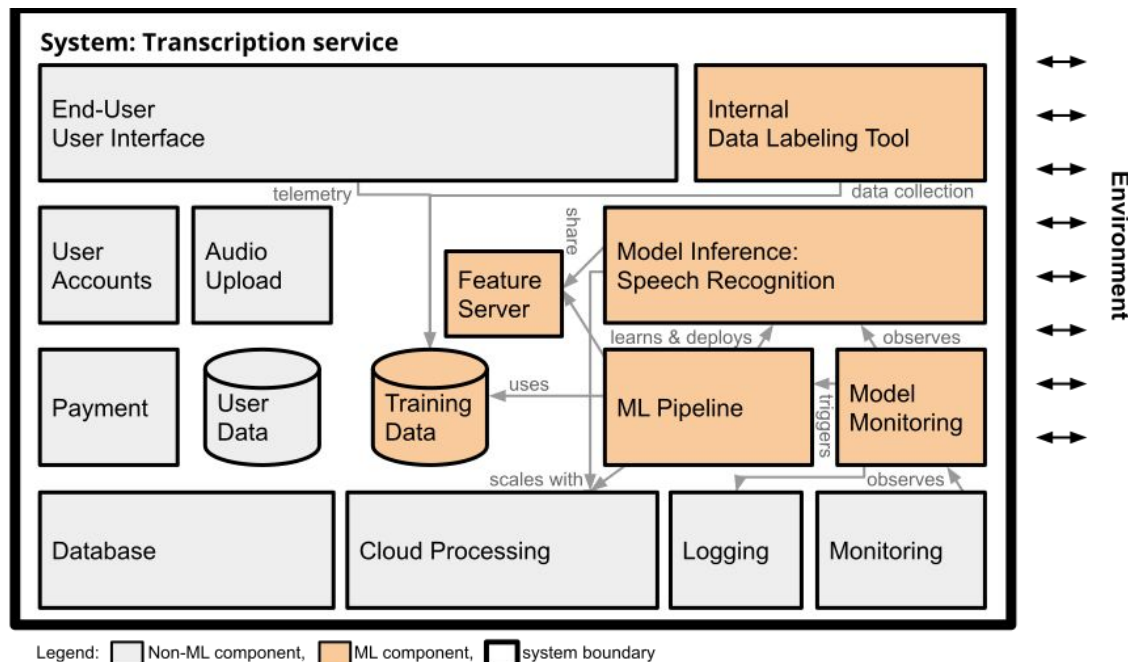
Requirements: Median reply under 30 seconds, under \$0.02 per response, wrong answers can have consequences

As a group, post and follow up in #lecture, tagging group members and @mlipbot:

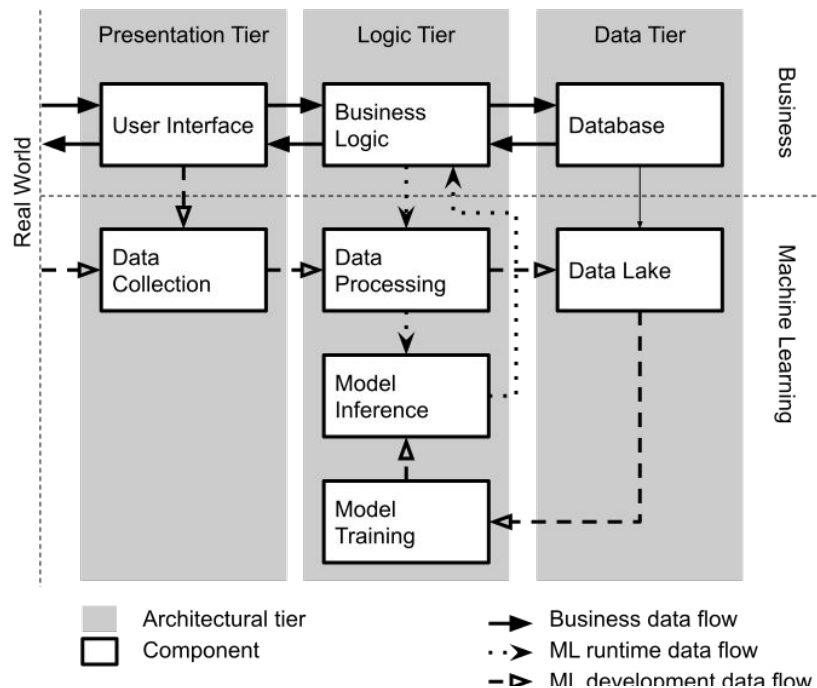
Sketch the architecture and name which patterns to use

AI Components within a Systems

Recall: Inference is a Component within a System



Separating Models and Business Logic



Based on: Yokoyama, Haruki. "Machine learning system architectural pattern for improving operational stability." In Int'l Conf. Software Architecture Companion, pp. 267-274. IEEE, 2019.

Separating Models and Business Logic

Clearly divide responsibilities

Allows largely independent and parallel work, assuming stable interfaces

Plan location of non-ML safeguards and other processing logic

More Considerations for Deployment Decisions

Coupling of ML pipeline parts

Coupling with other parts of the system

Ability for different developers and analysts to collaborate

Support online experiments

Ability to monitor

Documenting Model Inference Interfaces

Why Documentation

Model inference between teams:

- Data scientists developing the model
- Other data scientists using the model, evolving the model
- Software engineers integrating the model as a component
- Operators managing model deployment

Will this model work for my problem?

What problems to anticipate?

Classic API Documentation

```
/**  
 * compute deductions based on provided adjusted  
 * gross income and expenses in customer data.  
 *  
 * see tax code 26 U.S. Code A.1.B, PART VI  
 */  
float computeDeductions(float agi, Expenses expenses);
```

What to document for models?



Documenting Input/Output Types for Inference Components

```
{  
  "mid": string,  
  "languageCode": string,  
  "name": string,  
  "score": number,  
  "boundingPoly": {  
    object (BoundingPoly)  
  }  
}
```

From Google's public [object detection API](#).

Documentation beyond I/O Types

Intended use cases, model capabilities and limitations

Supported target distribution (vs preconditions)

Accuracy (various measures), incl. slices, fairness

Latency, throughput, availability (service level agreements)

Model qualities such as explainability, robustness, calibration

Ethical considerations (fairness, safety, security, privacy)

Example for OCR model? How would you describe these?

Model Cards

- Proposal and template for documentation from Google
 - Intended use, out-of-scope use
 - Training and evaluation data
 - Considered demographic factors
 - Accuracy evaluations
 - Ethical considerations
- 1-2 page summary
- Focused on fairness
- Widely discussed, but not frequently adopted

Model Card - Toxicity in Text

Model Details

- The TOXICITY classifier provided by Perspective API [32], trained to predict the likelihood that a comment will be perceived as toxic.
- Convolutional Neural Network.
- Developed by Jigsaw in 2017.

Intended Use

- Intended to be used for a wide range of use cases such as supporting human moderation and providing feedback to comment authors.
- Not intended for fully automated moderation.
- Not intended to make judgments about specific individuals.

Factors

- Identity terms referencing frequently attacked groups, focusing on sexual orientation, gender identity, and race.

Metrics

- Pinned AUC, as presented in [11], which measures threshold-agnostic separability of toxic and non-toxic comments for each group, within the context of a background distribution of other groups.

Ethical Considerations

- Following [31], the Perspective API uses a set of values to guide their work. These values are Community, Transparency, Inclusivity, Privacy, and Topic-neutrality. Because of privacy considerations, the model does not take into account user history when making judgments about toxicity.

Training Data

- Proprietary from Perspective API. Following details in [11] and [32], this includes comments from a online forums such as Wikipedia and New York Times, with crowdsourced labels of whether the comment is "toxic".
- "Toxic" is defined as "a rude, disrespectful, or unreasonable comment that is likely to make you leave a discussion."

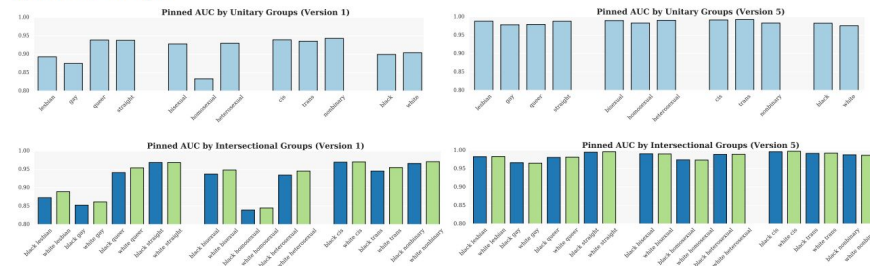
Evaluation Data

- A synthetic test set generated using a template-based approach, as suggested in [11], where identity terms are swapped into a variety of template sentences.
- Synthetic data is valuable here because [11] shows that real data often has disproportionate amounts of toxicity directed at specific groups. Synthetic data ensures that we evaluate on data that represents both toxic and non-toxic statements referencing a variety of groups.

Caveats and Recommendations

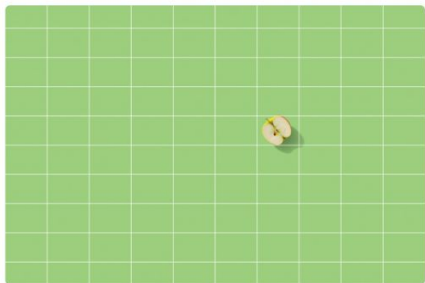
- Synthetic test data covers only a small set of very specific comments. While these are designed to be representative of common use cases and concerns, it is not comprehensive.

Quantitative Analyses



Limitations

The following factors may degrade the model's performance.



Object size: Object size must be at least 1% of the image area to be detected.



"Things" vs "stuff": Model was designed to detect discrete objects with clearly discernible shapes ("things"), not a group of overlapping objects or background clutter ("stuff").



Lighting: Poor or harsh, high-contrast illumination (e.g. nighttime, back-lit, side-lit) may degrade model performance.



Occlusion or clutter: Partially obstructed or truncated objects may not be detected. For example, a shirt underneath a jacket, or where less than 25% of an object is visible in the image.



Camera positioning and lens type: Camera angle and positioning (e.g. oblique angles, long-distance), and lens type (e.g. fisheye) may impact model performance.



Blur or noise: Blurry objects, rapid movement between frames, or encoding/decoding noise may degrade model performance.

FactSheets

Proposal and template for documentation from IBM; intended to communicate intended qualities and assurances

Longer list of criteria, including

- Service intention, intended use
- Technical description
- Target distribution
- Own and third-party evaluation results
- Safety and fairness considerations, explainability
- Preparation for drift and evolution
- Security, lineage and versioning

Documenting Models for Agents?

Recall: Correctness vs Fit

Without a clear specification a model is difficult to document

Need documentation to allow evaluation for *fit*

Description of *target distribution* is a key challenge

Summary

Model deployment seems easy, but involves many design decisions

- What models to use?
- Where to deploy?
- How to design feature encoding and feature engineering?
- How to compose with other components?
- How to document?
- How to collect telemetry?

Problem-specific modeling and analysis: Gather estimates, consider design alternatives, make tradeoffs explicit

Further Readings

Lakshmanan, Valliappa, Sara Robinson, and Michael Munn. Machine learning design patterns. O'Reilly Media, 2020.

Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. “Model cards for model reporting.” In Proceedings of the conference on fairness, accountability, and transparency, pp. 220–229. 2019.

Arnold, Matthew, Rachel KE Bellamy, Michael Hind, Stephanie Houde, Sameep Mehta, Aleksandra Mojsilović, Ravi Nair, Karthikeyan Natesan Ramamurthy, Darrell Reimer, Alexandra Olteanu, David Piorkowski, Jason Tsay, and Kush R. Varshney. “FactSheets: Increasing trust in AI services through supplier’s declarations of conformity.” IBM Journal of Research and Development 63, no. 4/5 (2019): 6–1.

Yokoyama, Haruki. “Machine learning system architectural pattern for improving operational stability.” In 2019 IEEE International Conference on Software Architecture Companion (ICSA-C), pp. 267–274. IEEE, 2019.